

# Clase 325 — Forense de memoria avanzado

Parte: **17 — Profundización para certificaciones** · Fuente: *The Art of Memory Forensics* (Ligh, Case, Levy, Walters) · SANS FOR508/FOR526 🕒 Duración estimada: **150 min** · Nivel: **Experto**

## Objetivo

Dominar el análisis forense de memoria RAM con **Volatility 3** como herramienta central: desde la adquisición correcta de un volcado hasta la caza de inyección de código, procesos ocultos, hooks y rootkits que solo viven en memoria. Al terminar, el alumno sabrá reconstruir la actividad de un atacante que nunca tocó el disco (malware *fileless*) y respaldar sus hallazgos con evidencia reproducible, tal como se exige en la certificación **GCFA** y en el módulo forense de **BTL1**.

## Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Adquirir** un volcado de memoria válido en Windows y Linux preservando la cadena de custodia (hash, metadatos, orden de volatilidad).
2. **Enumerar** procesos, conexiones de red y módulos con `windows.pslist`, `windows.pstree`, `windows.netscan` y correlacionarlos entre sí.
3. **Detectar** inyección de código con `windows.malfind`, distinguiendo regiones `PAGE_EXECUTE_READWRITE` maliciosas de falsos positivos legítimos.
4. **Identificar** procesos ocultos y desenganche de listas (DKOM) comparando `pslist` frente a `psscan`.
5. **Extraer** binarios, DLLs y cadenas sospechosas de memoria ( `dumpfiles`, `pslist --dump` ) para análisis posterior.
6. **Redactar** una línea de tiempo de memoria integrable con el resto de la evidencia del incidente.

## Temas

| # | Tema                                                                                                       | Por qué importa                                                     |
|---|------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| 1 | Orden de volatilidad y adquisición                                                                         | Un volcado mal tomado invalida todo el análisis posterior           |
| 2 | Arquitectura de Volatility 3 (symbol tables, ISF)                                                          | V3 abandona los perfiles de V2; entender los símbolos evita fallos  |
| 3 | Enumeración de procesos ( <code>pslist</code> , <code>pstree</code> , <code>psscan</code> )                | Base para detectar ocultamiento y relaciones padre-hijo anómalas    |
| 4 | Conexiones de red en memoria ( <code>netscan</code> )                                                      | Revela C2 activo aunque el firewall no lo registrara                |
| 5 | Detección de inyección ( <code>malfind</code> , <code>hollowfind</code> )                                  | El malware moderno se ejecuta dentro de procesos legítimos          |
| 6 | Rootkits y DKOM ( <code>ssdt</code> , <code>modules</code> , <code>modscan</code> )                        | Manipulación del kernel que oculta artefactos al SO en vivo         |
| 7 | Extracción y triaje de artefactos ( <code>dumpfiles</code> , <code>handles</code> , <code>cmdline</code> ) | Convierte un hallazgo en IOC accionable                             |
| 8 | Correlación con timeline del incidente                                                                     | La memoria es una foto; hay que ubicarla en la secuencia del ataque |

## Definiciones y características

- **Volcado de memoria (memory dump):** copia byte a byte de la RAM física en un instante. Característica clave: contiene datos que nunca se escriben a disco (claves, procesos, comandos).
- **Orden de volatilidad (RFC 3227):** principio que obliga a capturar primero lo más efímero (RAM, caché) antes que lo persistente (disco). Característica clave: la memoria se altera con cada acción, incluso al observarla.
- **Symbol table / ISF (Intermediate Symbol Format):** en Volatility 3, JSON que mapea estructuras del kernel a offsets. Característica clave: sustituye a los "profiles" de V2 y se resuelve casi siempre en automático.
- **malfind** : plugin que localiza regiones de memoria privadas, ejecutables y sin respaldo en disco. Característica clave: revela shellcode inyectado y *process hollowing*.
- **DKOM (Direct Kernel Object Manipulation):** técnica de rootkit que desenlaza el objeto `EPROCESS` de la lista doblemente enlazada. Característica clave: oculta el proceso a `pslist` pero **no** a `psscan` (que barre el pool).
- **Process hollowing:** vaciar un proceso legítimo suspendido y sustituir su imagen por código malicioso. Característica clave: PID legítimo, ruta legítima, contenido malicioso.
- **Fileless malware:** código que se ejecuta solo en memoria (PowerShell reflectivo, inyección). Característica clave: no deja archivo en disco; la memoria es a menudo la **única** fuente de evidencia.

## Herramientas y preparación

 **Laboratorio aislado obligatorio.** Todo análisis de muestras potencialmente maliciosas se hace en una VM sin conexión a la red corporativa, con snapshots y, si se manipulan binarios vivos, en una red

*host-only* o desconectada. Nunca analices malware en tu equipo de trabajo.

- **Volatility 3** ( `pip install volatility3` ), motor de análisis principal.
- **Adquisición Windows:** WinPmem ( `winpmem_mini_x64.exe` ), FTK Imager, DumpIt, Magnet RAM Capture.
- **Adquisición Linux:** LiME (Linux Memory Extractor) + módulo kernel, o AVML de Microsoft.
- **Muestras de práctica seguras:** volcados públicos de retos (p. ej. los *memory samples* del wiki de Volatility, MemLabs, retos de BTL1/CyberDefenders). Evitan tener que infectar una máquina real.
- **Utilidades de apoyo:** `strings` , `yara` , `capa` , un editor hexadecimal y `sha256sum` para hashing.

Verifica la instalación:

```
python3 -m volatility3.cli --help
vol -f memoria.raw windows.info    # atajo si instalaste el entrypoint 'vol'
```

## Laboratorio guiado

Trabajaremos sobre un volcado Windows llamado `caso.raw` . Reemplaza el nombre por tu muestra de práctica.

1. **Preserva la evidencia.** Antes de tocar nada, calcula y registra el hash:

```
bash sha256sum caso.raw | tee caso.raw.sha256
```

2. **Identifica el perfil/entorno del volcado:**

```
bash vol -f caso.raw windows.info
```

Anota versión de kernel, arquitectura y hora del sistema: fija el marco temporal del análisis. 3. **Enumera procesos y su jerarquía:**

```
bash vol -f caso.raw windows.pslist > pslist.txt vol -f caso.raw windows.pstree > pstree.txt
```

Busca padres anómalos: `winword.exe` lanzando `powershell.exe` , o `services.exe` con hijos raros. 4.

**Detecta procesos ocultos** comparando el barrido de pool contra la lista enlazada:

```
bash vol -f caso.raw windows.psscan > psscan.txt diff <(awk '{print $3}' pslist.txt) <(awk '{print $3}' psscan.txt)
```

Un PID que aparece en `psscan` pero **no** en `pslist` es candidato a ocultamiento por DKOM. 5. **Revisa las conexiones de red en memoria:**

```
bash vol -f caso.raw windows.netscan | grep -Ei 'ESTABLISHED|LISTENING'
```

Correlaciona cada IP/puerto remoto con el PID de un proceso sospechoso del paso 3. 6. **Caza inyección de código:**

```
bash vol -f caso.raw windows.malfind --dump --output-dir malfind_out
```

Inspecciona cada región marcada: la presencia de la cabecera `MZ` o de bytes de shellcode ( `\x55\x8b\xec` , `\xfc\xe8` ) en memoria `RWX` privada es una fuerte señal de inyección. 7. **Recupera la línea de comandos**

de los procesos sospechosos:

```
bash vol -f caso.raw windows.cmdline
```

PowerShell con `-enc`, `-nop`, `-w hidden` o Base64 largo indica ejecución ofuscada. 8. **Busca rootkits en el kernel:**

```
bash vol -f caso.raw windows.modules > modules.txt vol -f caso.raw windows.modscan > modscan.txt vol -f caso.raw windows.ssdt | grep -vi 'ntoskrnl\|win32k'
```

Un driver en `modscan` ausente de `modules`, o entradas SSDT que apuntan fuera de módulos legítimos, delatan un rootkit. 9. **Extrae los artefactos** para análisis estático posterior:

```
bash vol -f caso.raw windows.dumpfiles --pid <PID_SOSPECHOSO> --dump-dir extraidos yara reglas_malware.yar extraidos/ # aplica tus reglas YARA
```

10. **Construye la mini-timeline de memoria** y correlaciónala con el incidente:

```
bash vol -f caso.raw timeliner.Timeliner > timeline_mem.body
```

Cruza estas marcas con logs de EDR/Sysmon para ubicar el momento de la inyección dentro de la secuencia del ataque.

## Ejercicios

---

1. Adquiere un volcado de tu propia VM de práctica con WinPmem y verifica su hash antes y después de copiarlo.
2. En una muestra de práctica, lista los tres procesos con padre más sospechoso y justifica por qué usando `pstree`.
3. Ejecuta `malfind` y clasifica cada hit como *probable inyección* o *falso positivo*, explicando el criterio.
4. Detecta al menos un proceso oculto comparando `pslist` y `psscan`, y documenta el PID.
5. Extrae con `dumpfiles` el binario de un proceso malicioso y calcula su SHA-256 para búsqueda en VirusTotal (sin subir la muestra si es confidencial).
6. Genera una timeline con `timeliner.Timeliner` y correlaciona una conexión de `netscan` con su proceso y su hora de creación.

## Reto verificable

---

Recibes el volcado `incidente.raw` de un host presuntamente comprometido con malware *fileless*. **Criterio de aceptación:** entregas un informe corto que (a) identifica el PID y nombre del proceso inyectado, (b) muestra la salida de `malfind` que lo prueba, (c) lista la IP/puerto de C2 obtenida de `netscan` correlacionada con ese PID, y (d) incluye el SHA-256 del artefacto extraído con `dumpfiles`. El reto se considera logrado si otro analista puede reproducir tus hallazgos ejecutando los mismos comandos sobre el mismo volcado.

## ⚠ Errores comunes

| Síntoma / mensaje                                               | Causa y cómo arreglar                                                                                                                                               |
|-----------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Unable to validate the plugin requirements / no symbols         | El ISF del kernel no se resolvió; actualiza <code>volatility3</code> y sus <i>symbol packs</i> , o descarga el pack de símbolos correspondiente al build de Windows |
| <code>pslist</code> no muestra un proceso que sabes que existía | Ocultamiento por DKOM; usa <code>psscan</code> que barre el pool en vez de seguir la lista enlazada                                                                 |
| <code>malfind</code> arroja decenas de hits                     | Muchos son legítimos (JIT de navegadores, .NET); filtra por regiones <code>RWX</code> privadas con contenido tipo <code>MZ</code> /shellcode                        |
| Volcado tomado con la VM en ejecución da datos inconsistentes   | Se capturó mientras la memoria cambiaba; para VMs prefiere pausar y usar el archivo <code>.vmem</code> /snapshot                                                    |
| <code>netscan</code> no muestra conexiones esperadas            | Kernel muy nuevo o pool reciclado; complementa con artefactos de disco (Sysmon Event ID 3)                                                                          |
| Hashes distintos del mismo volcado en dos equipos               | Copia incompleta o corrupción de transferencia; re-copia y verifica cadena de custodia                                                                              |

## ? Preguntas frecuentes

? **¿Por qué Volatility 3 ya no usa "profiles"?** V3 resuelve la estructura del kernel mediante *symbol tables* (ISF) descargadas o generadas automáticamente a partir del build del SO, eliminando el paso manual de elegir profile que era fuente de errores en V2.

? **¿La adquisición altera la memoria?** Sí, mínimamente: la propia herramienta ocupa RAM. Por eso se documenta la herramienta usada y se respeta el orden de volatilidad, capturando la RAM antes que cualquier acción intrusiva.

? **¿malfind prueba por sí solo que hay malware?** No. Marca regiones sospechosas de inyección, pero hay falsos positivos legítimos. Confírmalo con el contenido de la región, la reputación del proceso y correlación con red y comandos.

? **¿Puedo analizar un volcado sin conocer la versión exacta de Windows?** `windows.info` la deduce del propio volcado. Si los símbolos no cargan, necesitarás el *symbol pack* del build específico.

? **¿Sirve la memoria si el atacante ya reinició la máquina?** No para ese instante: al apagar se pierde la RAM. Por eso la captura de memoria es prioritaria en la respuesta inicial (*live response*), antes de apagar o aislar el host.

## 🔗 Referencias

- Ligh, Case, Levy, Walters — *The Art of Memory Forensics* (Wiley, 2014).
- Documentación oficial de Volatility 3: <https://volatility3.readthedocs.io/>
- SANS FOR508 *Advanced Incident Response, Threat Hunting and Digital Forensics* y FOR526 *Memory Forensics In-Depth*.
- MITRE ATT&CK — Process Injection (T1055): <https://attack.mitre.org/techniques/T1055/>
- RFC 3227 — *Guidelines for Evidence Collection and Archiving*: <https://www.rfc-editor.org/rfc/rfc3227>

## **Siguiente clase**

---

Clase 326 - Análisis de malware para respuesta a incidentes